

CSS 430 Operating Systems

Program 4B: File Caching – Second-Chance Algorithm

1. Purpose

This assignment helps you understand the replacement mechanism and its performance improvement by implementing a buffer cache that stores frequently-accessed disk blocks onto the memory. To be more specific, you will implement a second-chance algorithm for your buffer cache, measure its performance when running various test cases, and consider the merits and demerits of your implementation.

2. Disk Caching

The idea of caching data from the slower to the faster storage is based on spatial and temporal locality of data accesses. User programs are likely to access the same data or the ones very close to the data previously accessed. This is called spatial locality and is the main reason why the operating system reads a block data or a page from the disk rather than repeats reading a few bytes of data the program has just requested. User programs also tend to access the same data repeatedly in a very short period of time which in turn means that the least-recently used blocks or pages are unlikely to be used and could be victims for replacement.

To accelerate disk accesses, we want to implement a disk cache that stores frequently-accessed disk blocks onto the memory, so that the second or the later accesses to the same disk block simply reads the data cached on the memory. When the disk cache is full of cached blocks, it must find out a victim to be replaced with a newly cached block. We will use *second-chance algorithm* (as described in our textbook.) To maintain data consistency between disk and cached blocks, *entry.h* defines each cached block as follows:

```
class Entry {
public:
    bool reference = false;    // true whenever this block as accessed. false by 2nd chance alg.
    bool dirty = false;       // true whenever this block is written. false upon a writeback.
    int pNumber = -1;         // contains the disk block number. -1 if this entry has no block.
};
```

The following instructions summarizes how to read, write, cache and replace a block of data:

- Block-read algorithm**
Scan all the cache entries. (Of course, sequential search is enough.) If the corresponding entry has been found in the cache, read the contents from the cache. Otherwise, find a free block entry in the cache and read the data from the disk to this cache block. If you cannot find out any free cache block, find a victim using the *second-chance algorithm*. If this victim is dirty, you must first write back its contents to the disk and thereafter read new data from the disk into this cache block. Do not forget to set the reference bit.
- Block-write algorithm**
Scan all the cache entries. (Again, sequential search is enough.) If the corresponding entry has been found in the cache, write new data to this cache block. Otherwise, find a free block entry in the cache and write the data to this cache block. You do not have to write this data through to the disk device. Mark its cache entry as dirty. If you cannot find out any free cache block, find a victim using the *second-chance algorithm*. If this victim is dirty, you must first write back its contents to the disk and thereafter write new data into this cache block. Do not forget to set the reference bit.
- Block-replacement algorithm**
Follow the *second-chance algorithm* described on your textbook.

These operations are defined in *file_cache.h*.

```
using namespace std;

class FileCache {
public:
    FileCache( );
    FileCache( string filename, int nLines );
    ~FileCache( );
    char *readPage( int pNumber );
    bool writePage( int pNumber, char contents[] );
    void sync( );
    void flush( );
    int getNPages( );    // return the total # of pages that fit a given file.
private:
    const char* filename; // a file name to read and write
    int fd;               // a file descriptor to access this file
    int nLines;           // the number of cache lines
    int victim;           // a victim cache line
    off_t page_len;       // the system memory page length: 4096 bytes
};
```

CSS 430 Operating Systems

Program 4B: File Caching – Second-Chance Algorithm

```
long file_len;           // the size of a file in bytes
Entry *entries;          // an array of disk cache entries
char **lines;            // actual disk cache lines

void init( string filename, int nLines ); // open a file
int findFreeEntry( );
int nextVictim( );
void writeBack( int victimEntry );
};
```

All method names are quite descriptive. Among these methods, your work in task 1 is implementing `readPage()`, `writePage()`, `findFreeEntry()`, `nextVictim()`, and `writeback()` in `file_cache.cpp`.

3. Memory-Mapped File versus Conventional File Operations

We will use Linux-supported memory-mapped file mechanism to read a bulk of file contents into a memory page (4096 bytes in general) rather than read a file with the conventional file `read()` system call. This is because Linux OS itself has a disk-caching mechanism at the kernel level. The conventional `read()` system call reads file data from its corresponding disk block, (in 512 bytes in general) into the kernel buffer space, and thereafter copies a user-specified length of data into a user memory space. So, implementing a disk cache using conventional `read()` and `write()` system calls wouldn't make sense as caching data twice.

A memory-mapped file allows a user to read disk data directly into a user-level memory space by skipping the kernel disk cache. An entire file can be read into memory with `mmap()` as far as its size fits memory space and the rest of file operations can be done with memory assignments or string operations without using conventional `read()` and `write()`. In `mmap()`, however such data transfers between disk and memory must be performed per memory pages, (a multiple of page size: $4096 * n$).

In accordance with this page-based data transfer scheme, we will cache 4096-byte disk data into one of `lines[]` (= `new char *[nLines]`) defined in `file_cache.h` and instantiated in `file_cache.cpp`'s `init()`. To read a 4096-byte disk block, we call:

```
// read a requested page from disk
lines[i] = (char*)mmap( lines[i], page_len, PROT_READ | PROT_WRITE, MAP_SHARED,
                        fd, pNumber * page_len );
```

where

`page_len`: the page size which is 4096 bytes

`fd`: a file descriptor to access this file

`pNumber`: a 0-index page number in this file to access

`pNumber * page_len`: the actual byte position in this file to access

We will use `msync()` for writing data from `line[i]` in `file_cache.cpp` back to disk as follows:

```
msync( lines[i], page_len, MS_SYNC );
```

Note that `msync()` only writes data back to disk but still maintains old data in `line[i]`. Therefore, whenever we reuse a cache line, say `line[i]`, we have to call `munmap()` as follows:

```
munmap( lines[i], page_len );
```

4. Code Structure

The 4B folder includes the following files:

File names	Descriptions
driver.cpp	Receives parameters as follows: <pre>./driver filename e nlines testcase</pre> where <code>e</code> = enabled or other = disabled <code>nlines</code> = #lines in cache (≥ 10) <code>testcase</code> = 1: random, 2: local, 3: mixed, 4: all Then, <code>main()</code> instantiate a Benchmark object. No need to change.
benchmark.h benchmark.cpp	Instantiated from <code>main()</code> , this class measure the performance of random, local, mixed, and all disk block accesses when disk caching is enabled or disabled. If disk caching is enabled, <code>filecache</code> is instantiated from the <code>FileCache</code> class that accesses a given file as a memory-mapped file and caches recently accessed cache line (or blocks) in <code>lines[]</code> . Otherwise, <code>barefile</code> is instantiated from the <code>BareFile</code> class that accesses a given file

CSS 430 Operating Systems

Program 4B: File Caching – Second-Chance Algorithm

	with conventional read() and write(). random-access(), localizedAccess(), and mixedAccess() create a pattern of 200 disk page accesses, each randomly accessing a different page, accessing pages to fit them in cache lines, and mixing 80% localized and 20% random accesses. No need to change.
bare_file.h bare_file.cpp	Opens a given file and read a given page into buffer[] or write contents[] into a given page of the file, using conventional read() and write() system calls. No need to change
file_cache.h	Defines a file cache entry and cache lines (or bocks) as well as readPage() and writePage() specifications. No need to change
file_cache.cpp	Implements a file cache. <pre>void FileCache::init(string filename, int nLines)</pre> opens a file in a read/write mode. If it doesn't exit, create a file. It creates a file cache with entries[nNines] and lines[nLines]. Already implemented. No need to change. <pre>int FileCache::findFreeEntry()</pre> Simply scans entries[] from top to down to search for an available entry. Implement by yourself. <pre>int FileCache::nextVictim()</pre> Finds a next victim, based on the second chance algorithm. Implement by yourself. <pre>void FileCache::writeBack(int victimEntry)</pre> Writes back victimEntry if it's dirty. Use mysnc(). Regardless victimEntry's dirty == true or false, you need to call munmap() to relinquish a previous memory page. Implement by yourself. <pre>void FileCache::sync()</pre> Writes back all cache lines. Already implemented. No need to change. <pre>void FileCache::flush()</pre> Writes back all cache lines. Already implemented as well as invalidates all cache lines. Already implemented. No need to change. <pre>int FileCache::getNPages()</pre> Returns the total number of pages that fit a given file. Already implemented. No need to change. <pre>char *FileCache::readPage(int pNumber)</pre> Read a page of data from disk into a disk cache, using mmap(). Implement by yourself. <pre>bool FileCache::writePage(int pNumber, char contents[])</pre> Write a page of data to a disk cache. If necessary, it calls writeback(). Implement by yourself.

5. Statement of Work

Task 1: Implement *file_cache.cpp* in accordance with the above specification by completing findFreeEntry(), nextVictim(), writeback(), readPage(), and writePage() of *file_cache_template.cpp*.

Task 2: Compile *driver.cpp*, *benchmark.cpp*, *bare_file.cpp*, and *file_cache.cpp* into *driver* and run the program as follows:

```
[css430@cssmpi1 4B]$ ./compile.sh
[css430@cssmpi1 4B]$ ./driver test.txt e 10 4
page length: 4096
file length: 2167737
Test random access(cache enabled): 0.041999 sec
Test localized access(cache enabled): 0.006387 sec
Test mixed access(cache enabled): 0.028854 sec
```

CSS 430 Operating Systems

Program 4B: File Caching – Second-Chance Algorithm

```
[css430@cssmpi1 4B]$ ./driver test.txt d 10 4
page length: 4096
file length: 2167737
Test random access(cache disabled): 0.010741 sec
Test localized access(cache disabled): 0.004322 sec
Test mixed access(cache disabled): 0.004448 sec
[css430@cssmpi1 4B]$ ./driver test.txt e 200 4
page length: 4096
file length: 2167737
Test random access(cache enabled): 0.009956 sec
Test localized access(cache enabled): 0.023665 sec
Test mixed access(cache enabled): 0.015808 sec
[css430@cssmpi1 4B]$ ./driver test.txt d 200 4
page length: 4096
file length: 2167737
Test random access(cache disabled): 0.009801 sec
Test localized access(cache disabled): 0.004641 sec
Test mixed access(cache disabled): 0.004415 sec
[css430@cssmpi1 4B]$ ./driver test.txt e 400 4
page length: 4096
file length: 2167737
Test random access(cache enabled): 0.009477 sec
Test localized access(cache enabled): 0.005739 sec
Test mixed access(cache enabled): 0.004331 sec
[css430@cssmpi1 4B]$ ./driver test.txt d 400 4
page length: 4096
file length: 2167737
Test random access(cache disabled): 0.009755 sec
Test localized access(cache disabled): 0.004460 sec
Test mixed access(cache disabled): 0.004379 sec
[css430@cssmpi1 4B]$ ./driver test.txt e 800 4
page length: 4096
file length: 2167737
Test random access(cache enabled): 0.009241 sec
Test localized access(cache enabled): 0.005893 sec
Test mixed access(cache enabled): 0.004670 sec
[css430@cssmpi1 4B]$ ./driver test.txt d 800 4
page length: 4096
file length: 2167737
Test random access(cache disabled): 0.012511 sec
Test localized access(cache disabled): 0.005192 sec
Test mixed access(cache disabled): 0.004374 sec
[css430@cssmpi1 4B]$
```

Task 3: Give a brief consideration about your performance comparisons between your file cache implementation and bare file accesses.

6. What to Turn in

	Materials	Points
1	Task 1: Your file_cache.cpp - Code organization: +2pts - Well organized: 2pts, - Poor comments or bad organization: 1pt, or - No comments and horrible code: 0pts - Correctness: +13pts - Reading from or writing to a cached line on a cache hit: +1pt - memcpy() use for copying a user page contents to the corresponding cache line: +1pt - The cached line's reference bit set upon a cache hit: +1pt - The cached line's dirty bit set upon a write: +1pt - An invalid cached line searched for on a cache miss: +2pts - A victim chosen after no invalid cache line is found: +2pts - The victim written back to disk if it is dirty: +2pts - The victim's dirty bit reset upon a write back: +1pt - A cache line read from disk into the invalid or victim block: +2pts, and	16

CSS 430 Operating Systems
Program 4B: File Caching – Second-Chance Algorithm

	• The cache line's page# and reference bit updated upon a disk read: +1pt	
2	Task 2: Your execution output: +2pts • Any performance improvements with cache enabled: 2pts, • No performance improvements with cache enabled: 1pt, or • No outputs: 0pts	2
3	Task 3: Give a brief consideration on your measurements: +2pts • Reasonable consideration/justification: 2pts, • Wrong consideration: 1pt, or • No discussions at all: 0pts	2

Total 20pts.